
THIMBLE

Developer Implementation Guide

STATUS

Phase 1 Ready

TIMELINE

4 Weeks → 12+ Weeks

PHASES

4 Phases

FEATURES

Likes, Comments, Follows, Saves, Feed Algorithm

DATE

May 11, 2026

Keep this guide updated as items are completed

AUDIT SUMMARY

Issue	Count	Priority
Broken / Fake Pages	1	CRITICAL
Pages with Mock Fallback	3	CRITICAL
Non-functional Buttons	14+	CRITICAL
Missing Social Features	4 (likes, comments, follows, saves)	CRITICAL
Mock Data Arrays to Delete	3	HIGH
Hardcoded Stats	7	HIGH
Missing Backend Routes	8	HIGH

Broken Items Found in Codebase

- X Explore page is 100% hardcoded (9 fake posts, 7 fake categories)
- X Right rail: fake stats, fake trending tags, fake suggested users
- X Sidebar: fake trending topics, non-functional follow buttons
- X Like button: toggles local state only, never hits database
- X Save button: local state only, no API call
- X Comments: hardcoded count always shows 18
- X Gig apply: increments browser memory, resets on refresh
- X Post composer: `onClick={() {}}` does nothing
- X Follow buttons: no `onClick` handler at all (6+ buttons)
- X Profile followers/following: always shows 0

IMPLEMENTATION PHASES

PHASE 1: Backend First

Timeline: Week 1

- 5 new database tables (likes, comments, follows, saves, applications)
- 8 new API routes (like/comment/follow/save/apply/feed/explore/suggest)
- Engagement-based feed algorithm
- Do NOT touch frontend yet

PHASE 2: Clean Frontend

Timeline: Week 2-3

- Delete all mock data from `lib/store.ts`
- Wire every button to real API
- Fix hardcoded stats (followers, comments count, etc.)
- Remove fallback to mock data
- Fix explore page, right rail, sidebar

PHASE 3: New Features

Timeline: Week 4

- Public profile pages
- Notifications system
- S3 blob storage (ESSENTIAL for image uploads)
- Rate limiting on auth routes

PHASE 4: Polish

Timeline: Month 2-3

- Hashtags / trending tags
- Search functionality
- Image optimization
- Advanced UX improvements

WEEK-BY-WEEK BREAKDOWN

Week 1: Backend Infrastructure

- Add 5 DB tables: `post_likes`, `post_comments`, `follows`, `saved_posts`, `gig_applications`
- Add 8 backend API routes (Go code provided in guide)
- Test each route with `curl` or Postman

Week 2: Data Cleanup + First Features

- Delete `mockGigs` array from `lib/store.ts`
- Delete `mockDesignPosts` array from `lib/store.ts`
- Delete hardcoded `explore posts` array
- Wire like button to `/api/posts/:id/like`
- Wire follow buttons to `/api/users/:id/follow`

Week 3: More Frontend Wiring

- Wire comments UI to `/api/posts/:id/comments`
- Wire save button to `/api/posts/:id/save`
- Fix profile followers/following count (use real data from user object)
- Fix gig apply button to `/api/gigs/:id/apply`
- Fix post composer (Photo, Gig buttons should open modals)

Week 4: New Pages + Security

- Build public profile pages (`/profile/[userId]`)
- Fix explore page to use `/api/explore` (not hardcoded)
- Add rate limiting to auth endpoints
- Add feed mode toggle (For You / Following / Latest)
- Add input sanitization to prevent XSS

NEW DATABASE TABLES

post_likes

```
CREATE TABLE post_likes (  
  id          BIGSERIAL PRIMARY KEY,  
  post_id     BIGINT NOT NULL,  
  user_id     TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(post_id, user_id)  
);
```

post_comments

```
CREATE TABLE post_comments (  
  id          BIGSERIAL PRIMARY KEY,  
  post_id     BIGINT NOT NULL,  
  user_id     TEXT NOT NULL,  
  user_name   TEXT NOT NULL,  
  user_avatar TEXT DEFAULT '',  
  content     TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

follows

```
CREATE TABLE follows (  
  id          BIGSERIAL PRIMARY KEY,  
  follower_id TEXT NOT NULL,  
  followed_id TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(follower_id, followed_id)  
);
```

saved_posts

```
CREATE TABLE saved_posts (  
  id          BIGSERIAL PRIMARY KEY,  
  post_id     BIGINT NOT NULL,  
  user_id     TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(post_id, user_id)  
);
```

gig_applications

```
CREATE TABLE gig_applications (  
  id          BIGSERIAL PRIMARY KEY,  
  gig_id      BIGINT NOT NULL,  
  user_id     TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(gig_id, user_id)  
);
```

NEW API ROUTES (8 Total)

Endpoint	Purpose
POST /api/posts/:id/like	Toggle like on a post
GET /api/posts/liked	Get all post IDs the current user liked
GET /api/posts/:id/comments	Get all comments on a post
POST /api/posts/:id/comments	Create a comment on a post
DELETE /api/posts/:id/comments/:commentId	Delete a comment
POST /api/users/:id/follow	Toggle follow on a user
GET /api/users/:id/followers	Get followers of a user
GET /api/users/:id/following	Get users that a user follows
POST /api/posts/:id/save	Toggle save a post
GET /api/posts/saved	Get current user's saved posts
POST /api/gigs/:id/apply	Toggle apply to a gig
GET /api/explore	Get feed for discover page
GET /api/users/suggested	Get suggested users to follow

Full Go code for all 13 routes is in the developer guide.

WHAT NOT TO BUILD (Zero Budget)

These features are expensive and can wait until there is budget:

Mobile App: 6+ months, requires React Native or Swift/Kotlin expertise

Video Uploads: S3 + transcoding costs multiply; need CDN bandwidth

AI-Powered Feed: Requires ML infrastructure; data science team needed

Stories / Highlights: Complex media pipeline; high storage + bandwidth

Paid Jobs (Escrow): Requires Stripe + legal; payment disputes are costly

SMS 2FA: Costs per SMS; TOTP (authenticator app) is free

Advanced Creator Analytics: Requires event tracking pipeline; big project

TESTING REQUIREMENTS

Every feature must have tests before it is marked done:

- Like/Unlike toggle (happy path + double like = unlike)
- Comments CRUD (create, read, delete only by author)
- Follow/Unfollow toggle (can't follow yourself)
- Save/Unsave toggle
- Gig applications (can't apply twice)
- Feed algorithm (engagement-based ranking works)
- Unauthenticated access (should return 401)

NEXT STEPS

1. Print or save this PDF
2. Read `/docs/DEVELOPER_GUIDE.md` (full implementation details)
3. Start Phase 1: Add database tables to `backend/main.go`
4. Test each backend route with `curl` before touching frontend
5. Follow the week-by-week breakdown in order
6. Check off items as completed

Full developer guide: `/docs/DEVELOPER_GUIDE.md`

Generated: May 11, 2026